

Understanding Flow Matching from a mathematical view

Pipi Hu

Beijing Institute of Mathematical Sciences and Applications

November 11, 2025

Outline

- 1 Mathematical motivation for Flow Matching
- 2 Connection with Diffusion Models
- 3 Conditional Paths and Velocity Targets
- 4 Flow Matching Training and Inference
- 5 Advanced Topics and Extensions
- 6 MeanFlow: Single-Step Generation
- 7 Summary and Future Directions
- 8 References

Outline

- 1 Mathematical motivation for Flow Matching
- 2 Connection with Diffusion Models
- 3 Conditional Paths and Velocity Targets
- 4 Flow Matching Training and Inference
- 5 Advanced Topics and Extensions
- 6 MeanFlow: Single-Step Generation
- 7 Summary and Future Directions
- 8 References

Probability Path and Coupling

- Let $\{p_t\}_{t \in [0,1]}$ be a prescribed path of marginals on $\mathbb{R}^d$ with p_0 simple and p_1 the data.
- A *coupling* induces random trajectories $t \mapsto x_t$ with instantaneous velocity $\dot{x}_t = \frac{d}{dt}x_t$ (may be stochastic given x_t).
- Goal: learn a *deterministic* vector field $v : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d$ such that the ODE

$$\frac{d}{dt}x_t = v(x_t, t), \quad x_0 \sim p_0,$$

yields marginals $x_t \sim p_t$ for all t .

Remark

We can depict the flow matching and diffusion through the probability transition v.s. the trajectories. And also, we can connect the Neural ODE with the Flow matching.

Strong and Weak Continuity

From the Fokker–Planck equation, for $\frac{dx}{dt} = v(x, t)$, we have

$$\partial_t p_t(x) + \nabla \cdot (p_t(x) v(x, t)) = 0.$$

The corresponding weak form of above strong form is

Weak form (for any test functions $\varphi \in C_c^\infty(\mathbb{R}^d)$)

$$\frac{d}{dt} \int_{\mathbb{R}^d} \varphi(x) p_t(x) dx = \int_{\mathbb{R}^d} \nabla \varphi(x) \cdot v(x, t) p_t(x) dx,$$

i.e.,

$$\frac{d}{dt} \mathbb{E}[\varphi(x_t)] = \mathbb{E}[\nabla \varphi(x_t) \cdot v(x, t)].$$

Weak Form from Random Trajectories

Let x_t be the coupled trajectory (not necessarily deterministic). For any test function $\phi \in C_c^\infty(\mathbb{R}^d)$, we have

$$\frac{d}{dt} \mathbb{E}[\varphi(x_t)] = \mathbb{E}[\nabla \varphi(x_t) \cdot \dot{x}_t],$$

by the expectation of the derivative of the test function.

Expanding the expectation into integrals at a fixed t :

$$\begin{aligned} \mathbb{E}[\nabla \varphi(x_t) \cdot \dot{x}_t] &= \iint_{\mathbb{R}^d \times \mathbb{R}^d} \nabla \varphi(x) \cdot \dot{x} \mu_t(dx, d\dot{x}) \\ &= \int_{\mathbb{R}^d} \nabla \varphi(x) \cdot \left(\int \dot{x} p(\dot{x} | x, t) d\dot{x} \right) p_t(x) dx \\ &= \int_{\mathbb{R}^d} \nabla \varphi(x) \cdot v^*(x, t) p_t(x) dx = \mathbb{E}[\nabla \varphi(x_t) \cdot v^*(x, t)]. \end{aligned}$$

where $v^*(x, t) \equiv \mathbb{E}[\dot{x}_t | x_t = x, t] = \int_{\mathbb{R}^d} \dot{x} p(\dot{x} | x, t) d\dot{x}$. Thus the random path induces the weak identity with $v = v^*$.

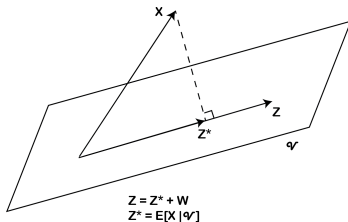
Theoretical Support: Conditional Expectation

Theorem

Let X be an integrable random variable. Then for each σ -algebra $\mathcal{V}$ and $Z \in \mathcal{V}$, $Z^* = \mathbb{E}(X|\mathcal{V})$ solves the least square problem

$$\operatorname{argmin}_{Z \in \mathcal{V}} \|Z - X\|,$$

where $\|Z\| = \left(\int Z^2 dP\right)^{\frac{1}{2}}$.



Hence we have $L = \mathbb{E}\|v_\theta(x_t, t) - \dot{x}_t\|^2$.

Practical Training Recipe (Per-Iteration)

- 1 Sample $t \sim w(t)$ (typically uniform on $[0, 1]$).
- 2 Sample $(x_t, \dot{x}_t)$ from the chosen path coupling.
- 3 Regress $\dot{x}_t$ on (x_t, t) via the MSE:

$$\mathcal{L} = \|v_\theta(x_t, t) - \dot{x}_t\|^2.$$

- 4 Update θ by SGD; no density estimation, no reverse simulation is needed during training.

Remark

Different path/coupling choices change $v^\star$ and the irreducible variance $\mathbb{E}\|\dot{x}_t - v^\star\|^2$, affecting sample efficiency and optimization, but the optimal target remains the conditional mean.

Key Takeaways

- The correct notion of “matching the path” is the **weak continuity** identity for all test functions.
- The only deterministic field depending on (x, t) that preserves this identity is the **conditional mean velocity**

$$v^*(x, t) = \mathbb{E}[\dot{x}_t \mid x_t = x, t].$$

- The **flow-matching MSE** is precisely the L^2 projection selecting v^* :

$$v_\theta \xrightarrow[\text{minimize}]{\mathbb{E} \|v_\theta(x_t, t) - \dot{x}_t\|^2} v^*.$$

Outline

- 1 Mathematical motivation for Flow Matching
- 2 Connection with Diffusion Models**
- 3 Conditional Paths and Velocity Targets
- 4 Flow Matching Training and Inference
- 5 Advanced Topics and Extensions
- 6 MeanFlow: Single-Step Generation
- 7 Summary and Future Directions
- 8 References

Theorem (Fokker–Planck / Forward Kolmogorov)

Let $(X_t)_{t \geq 0} \in \mathbb{R}^N$ solve the Itô SDE

$$dX_t = \mu(X_t, t) dt + \sigma(X_t, t) dW_t,$$

where $\mu : \mathbb{R}^N \times [0, \infty) \rightarrow \mathbb{R}^N$ is the drift, $\sigma : \mathbb{R}^N \times [0, \infty) \rightarrow \mathbb{R}^{N \times M}$ the diffusion, and W_t an M -dimensional standard Wiener process. Define the diffusion tensor

$$D(x, t) := \frac{1}{2} \sigma(x, t) \sigma(x, t)^\top \in \mathbb{R}^{N \times N}.$$

Then the (smooth) density $p(x, t)$ satisfies the Fokker–Planck equation

$$\partial_t p(x, t) = - \sum_{i=1}^N \frac{\partial}{\partial x_i} [\mu_i(x, t) p(x, t)] + \sum_{i,j=1}^N \frac{\partial^2}{\partial x_i \partial x_j} [D_{ij}(x, t) p(x, t)]$$

Isotropic diffusion: $\sigma(x, t) = \sigma(t) \mathbf{I}_N$

When $\sigma(x, t) = \sigma(t) \mathbf{I}_N$, the diffusion tensor becomes

$$D(x, t) = \frac{1}{2} \sigma(t)^2 \mathbf{I}_N, \quad D_{ij}(x, t) = \frac{1}{2} \sigma(t)^2 \delta_{ij}.$$

Hence the Fokker–Planck equation simplifies to the drift–diffusion form

$$\partial_t p(x, t) = -\nabla \cdot (\mu(x, t) p(x, t)) + \frac{\sigma(t)^2}{2} \Delta p(x, t)$$

where $\nabla \cdot (\mu p) = \sum_{i=1}^N \partial_{x_i} (\mu_i p)$ and $\Delta = \sum_{i=1}^N \partial_{x_i}^2$ is the Laplacian.

Mathematical connection to diffusion models

Theoretical link: For a forward SDE $d\mathbf{x} = f(\mathbf{x}, t) dt + g(t) dB_t$, the probability flow ODE is:

$$\frac{d}{dt} \mathbf{x} = f(\mathbf{x}, t) - \frac{1}{2} g^2(t) \nabla_{\mathbf{x}} \log p(\mathbf{x}, t). \quad (1)$$

Key mathematical insight: When Flow Matching uses Gaussian paths that match the SDE's marginal distributions p_t , the learned velocity field $\mathbf{v}_\theta$ approximates the probability flow drift:

$$\mathbf{v}_\theta(\mathbf{x}, t) \approx f(\mathbf{x}, t) - \frac{1}{2} g^2(t) \nabla_{\mathbf{x}} \log p(\mathbf{x}, t) \quad (2)$$

Crucial advantage: Flow Matching achieves this approximation *without* ever computing the score $\nabla_{\mathbf{x}} \log p(\mathbf{x}, t)$ during training!

Practical implications: This connection allows Flow Matching to inherit the theoretical guarantees of diffusion models while avoiding their computational bottlenecks.

Mathematical derivation: probability flow ODE

Setup: Forward SDE $d\mathbf{x} = f(\mathbf{x}, t) dt + g(t) dB_t$ with density $p(\mathbf{x}, t)$ solving the Fokker-Planck equation:

$$\partial_t p = -\nabla \cdot (fp) + \frac{1}{2}g^2 \Delta p. \quad (3)$$

Key insight: A deterministic ODE $\dot{\mathbf{x}} = \mathbf{v}(\mathbf{x}, t)$ transports densities via the continuity equation:

$$\partial_t p + \nabla \cdot (p\mathbf{v}) = 0. \quad (4)$$

Mathematical derivation: Equating the two equations:

$$-\nabla \cdot (p\mathbf{v}) = -\nabla \cdot (fp) + \frac{1}{2}g^2 \Delta p \quad (5)$$

$$\Rightarrow \mathbf{v} = f - \frac{1}{2}g^2 \frac{\nabla p}{p} = f - \frac{1}{2}g^2 \nabla \log p. \quad (6)$$

Result: The probability flow ODE drift is $\mathbf{v} = f - \frac{1}{2}g^2 \nabla \log p$.

Connection to Flow Matching: When Flow Matching learns $\mathbf{v}_\theta$, it approximates this probability flow drift without computing the score $\nabla \log p$.

Questions

From

- ① Score model $s_\theta(x_t) \rightarrow s_{\theta^*}(x_t) \equiv \mathbb{E}[\nabla \log p(x_t|x_0) | x_t]$
- ② X prediction model (denoising model, x_0 model) $x_\theta(x_t) \rightarrow x_{\theta^*}(x_t) \equiv \mathbb{E}[x_0 | x_t]$;
- ③ ϵ prediction model $\epsilon_\theta(x_t) \rightarrow \epsilon_{\theta^*}(x_t) \equiv \mathbb{E}[\epsilon | x_t]$.

and

$$v^*(x_t) = \mathbb{E}[\dot{x}_t | x_t]$$

, what is the relationship between optimal $v_\theta(x, t)$ and the above models? Hints:

$$v_{\theta^*}(x_t) = \mathbb{E}[\dot{x}_t | x_t] = \mathbb{E}[\dot{\alpha}(t)x_t + \dot{\sigma}(t)\epsilon | x_t]. \quad (7)$$

and

$$s(x_t) = \mathbb{E}_x[\nabla_{x_t} \log p(x_t|x_0)|x_t] = \mathbb{E}_x[-\frac{x_t - \alpha x_0}{\sigma^2} | x_t] = -\frac{x_t - \alpha \mathbb{E}[x_0|x_t]}{\sigma^2}.$$

Outline

- 1 Mathematical motivation for Flow Matching
- 2 Connection with Diffusion Models
- 3 Conditional Paths and Velocity Targets**
- 4 Flow Matching Training and Inference
- 5 Advanced Topics and Extensions
- 6 MeanFlow: Single-Step Generation
- 7 Summary and Future Directions
- 8 References

Gaussian conditional paths: mathematical derivation

Assumption: Gaussian conditional paths

$$q_t(\mathbf{x} \mid \mathbf{x}_0) = \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_t(\mathbf{x}_0), \sigma_t^2 \mathbf{I}), \quad \mathbf{x}_t = \boldsymbol{\mu}_t(\mathbf{x}_0) + \sigma_t \boldsymbol{\epsilon}. \quad (9)$$

Mathematical derivation: Taking the time derivative of the path:

$$\frac{d}{dt} \mathbf{x}_t = \dot{\boldsymbol{\mu}}_t(\mathbf{x}_0) + \dot{\sigma}_t \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} = \frac{\mathbf{x}_t - \boldsymbol{\mu}_t(\mathbf{x}_0)}{\sigma_t}. \quad (10)$$

Conditional expectation: Since $\boldsymbol{\epsilon}$ is independent of $\mathbf{x}_t$ given $\mathbf{x}_0$, we have:

$$\boxed{v = \dot{\boldsymbol{\mu}}_t(\mathbf{x}_0) + \frac{\dot{\sigma}_t}{\sigma_t} (\mathbf{x} - \boldsymbol{\mu}_t(\mathbf{x}_0))}. \quad (11)$$

Mathematical analysis of path choices

1. Linear bridge (optimal transport):

- $\mu_t(\mathbf{x}_0) = (1 - t) \mathbf{x}_0, \sigma_t = t \sigma_{\max}$
- **Mathematical property:** Minimizes Wasserstein-2 distance
- **Velocity:** $v = \frac{\sigma_{\max}}{t}(\mathbf{x} - \mathbf{x}_0)$
- **Advantage:** Direct connection to optimal transport theory

2. VP-like (DDPM-inspired):

- $\mu_t(\mathbf{x}_0) = \alpha(t) \mathbf{x}_0, \sigma_t = \sqrt{1 - \alpha^2(t)}$
- **Mathematical property:** Preserves energy structure
- **Velocity:** $v = \dot{\alpha}(t) \mathbf{x}_0 + \frac{\dot{\sigma}_t}{\sigma_t}(\mathbf{x} - \alpha(t) \mathbf{x}_0)$
- **Advantage:** Compatible with existing diffusion model schedules

3. VE-like (SMLD-inspired):

- $\mu_t(\mathbf{x}_0) = \mathbf{x}_0, \sigma_t = \sigma(t)$ increasing
- **Mathematical property:** Preserves data structure
- **Velocity:** $v = \frac{\dot{\sigma}(t)}{\sigma(t)}(\mathbf{x} - \mathbf{x}_0)$
- **Advantage:** Simple form, good for high-dimensional data

Key insight: All choices provide *closed-form* $\mathbf{u}_t$ via the boxed formula!

Derivation: VE/VP target forms

VE-like. $\mu_t(\mathbf{x}_0) = \mathbf{x}_0$, $\sigma_t = \sigma(t)$. Then

$$\mathbf{v} = 0 + \frac{\dot{\sigma}_t}{\sigma_t} (\mathbf{x} - \mathbf{x}_0). \quad (12)$$

VP-like. $\mu_t(\mathbf{x}_0) = \alpha(t) \mathbf{x}_0$, $\sigma_t = \sqrt{1 - \alpha^2(t)}$. Using $\mathbf{u}_t = \dot{\mu}_t + (\dot{\sigma}_t/\sigma_t)(\mathbf{x} - \mu_t)$, we get

$$\mathbf{v} = \dot{\alpha} \mathbf{x}_0 + \frac{\dot{\sigma}_t}{\sigma_t} \mathbf{x} - \frac{\dot{\sigma}_t}{\sigma_t} \alpha \mathbf{x}_0 \quad (13)$$

$$= \frac{\dot{\sigma}_t}{\sigma_t} \mathbf{x} + \left(\dot{\alpha} - \alpha \frac{\dot{\sigma}_t}{\sigma_t} \right) \mathbf{x}_0. \quad (14)$$

No explicit form for $\dot{\sigma}_t/\sigma_t$ is required to train; schedules determine it numerically.

Mathematical insight: These derivations show how different path choices lead to different velocity field structures, all computable analytically.

Outline

- 1 Mathematical motivation for Flow Matching
- 2 Connection with Diffusion Models
- 3 Conditional Paths and Velocity Targets
- 4 Flow Matching Training and Inference**
- 5 Advanced Topics and Extensions
- 6 MeanFlow: Single-Step Generation
- 7 Summary and Future Directions
- 8 References

Flow Matching: mathematical training objective

Theoretical foundation: Given conditional paths $q_t(\mathbf{x} \mid \mathbf{x}_0)$, the optimal velocity field satisfies:

$$\mathbf{v}^*(x, t) = \mathbb{E}[\dot{\mathbf{x}}_t \mid x_t = x, t] \quad (15)$$

Training procedure:

- 1 Sample $t \sim \mathcal{U}[0, 1]$, $\mathbf{x}_0 \sim p_{\text{data}}$, $\mathbf{x}_t \sim q_t(\cdot \mid \mathbf{x}_0)$
- 2 Compute analytic target $\mathbf{u}_t(\mathbf{x}_t \mid \mathbf{x}_0)$ from chosen path
- 3 Minimize regression loss

Mathematical objective:

$$J_{\text{FM}} = \mathbb{E}[\|\mathbf{v}_\theta(\mathbf{x}_t, t) - \mathbf{v}\|^2]. \quad (16)$$

Key mathematical advantage: No score estimation needed; all targets are analytic!

Computational benefits:

- Stable gradients (no exploding/vanishing)
- Parallelizable across time steps
- No special architectural requirements

Mathematical foundation of inference

Theoretical basis: The learned velocity field $\mathbf{v}_\theta(\mathbf{x}, t)$ defines a deterministic ODE:

$$\frac{d}{dt} \mathbf{x} = \mathbf{v}_\theta(\mathbf{x}, t), \quad t \in [0, 1] \quad (17)$$

Mathematical properties:

- **Existence:** Under Lipschitz conditions, solutions exist and are unique
- **Consistency:** The ODE preserves the continuity equation
- **Stability:** Deterministic integration avoids accumulation of stochastic errors

Generation process:

- 1 Initialize $\mathbf{x}(1) \sim p_1$ (e.g., $\mathcal{N}(0, \mathbf{I})$)
- 2 Integrate ODE backwards: $\frac{d}{dt} \mathbf{x} = \mathbf{v}_\theta(\mathbf{x}, t)$, $t : 1 \rightarrow 0$
- 3 Use standard ODE solvers (Euler, RK4, Dormand-Prince, etc.)

Mathematical advantages:

- Deterministic sampling (reproducible)
- Fast with large solver steps
- No stochastic correctors needed

Outline

- 1 Mathematical motivation for Flow Matching
- 2 Connection with Diffusion Models
- 3 Conditional Paths and Velocity Targets
- 4 Flow Matching Training and Inference
- 5 Advanced Topics and Extensions**
- 6 MeanFlow: Single-Step Generation
- 7 Summary and Future Directions
- 8 References

Conditional Flow Matching

Extension to conditional generation: Instead of unconditional paths, we can define conditional paths $q_t(\mathbf{x} \mid \mathbf{x}_0, c)$ where c represents conditioning information (e.g., class labels, text prompts).

Mathematical formulation: The conditional velocity field becomes:

$$\mathbf{u}_t(\mathbf{x} \mid \mathbf{x}_0, c) := \mathbb{E}\left[\frac{d}{dt} \mathbf{x}_t \mid \mathbf{x}_t = \mathbf{x}, \mathbf{x}_0, c\right] \quad (18)$$

Training objective:

$$J_{\text{CFM}} = \mathbb{E}\left[\|\mathbf{v}_\theta(\mathbf{x}_t, t, c) - \mathbf{v}\|^2\right] \quad (19)$$

Applications:

- Class-conditional generation
- Text-to-image synthesis
- Image-to-image translation
- Style transfer

Advantage: Maintains the same computational efficiency as unconditional Flow Matching while enabling controlled generation.

Rectified Flow and Optimal Transport

Rectified Flow: A special case of Flow Matching that uses straight-line paths:

$$\mathbf{x}_t = (1 - t)\mathbf{x}_0 + t\mathbf{x}_1, \quad \mathbf{v} = \mathbf{x}_1 - \mathbf{x}_0 \quad (20)$$

Mathematical properties:

- **Optimal transport:** Minimizes Wasserstein-2 distance
- **Simplicity:** Constant velocity field along each path
- **Efficiency:** Fastest convergence in many cases

Connection to optimal transport: Rectified Flow recovers the optimal transport map between p_0 and p_1 when the data is well-structured.

Practical advantages:

- Fewer function evaluations during sampling
- More stable training
- Better sample quality in many domains

Flow Matching in Latent Spaces

Motivation: Instead of working directly in pixel space, Flow Matching can be applied in learned latent representations.

Mathematical setup: Given an encoder $E : \mathcal{X} \rightarrow \mathcal{Z}$ and decoder $D : \mathcal{Z} \rightarrow \mathcal{X}$:

- Learn flow in latent space: $\mathbf{v}_\theta : \mathcal{Z} \times [0, 1] \rightarrow \mathcal{Z}$
- Generate latent codes: $\mathbf{z}_0 = \mathbf{z}_1 + \int_0^1 \mathbf{v}_\theta(\mathbf{z}_t, t) dt$
- Decode to images: $\mathbf{x} = D(\mathbf{z}_0)$

Advantages:

- **Computational efficiency:** Lower-dimensional space
- **Sample quality:** Better inductive biases in latent space
- **Flexibility:** Can use different architectures for encoder/decoder

Examples: Latent Diffusion Models (LDMs), Stable Diffusion

Outline

- 1 Mathematical motivation for Flow Matching
- 2 Connection with Diffusion Models
- 3 Conditional Paths and Velocity Targets
- 4 Flow Matching Training and Inference
- 5 Advanced Topics and Extensions
- 6 MeanFlow: Single-Step Generation**
- 7 Summary and Future Directions
- 8 References

MeanFlow: Motivation

Goal: Achieve single-step (1-NFE) generation while retaining high fidelity and stability.

Problem with standard Flow Matching: Multi-step ODE integration is computationally expensive and can accumulate errors.

Key insight: Learn a *ground-truth average velocity field* that summarizes an entire time interval.

Mathematical formulation: For any pair $r < t$ and point $\mathbf{z}_t$ along the flow path:

$$\bar{\mathbf{u}}(\mathbf{z}_t, r, t) := \frac{1}{t - r} \int_r^t \mathbf{v}(\mathbf{z}_\tau, \tau) \, d\tau. \quad (21)$$

Properties:

- $\bar{\mathbf{u}}$ is an **underlying field** induced by $\mathbf{v}$ (no networks yet)
- As $r \rightarrow t$, $\bar{\mathbf{u}}(\mathbf{z}_t, r, t) \rightarrow \mathbf{v}(\mathbf{z}_t, t)$
- Composition over intervals holds: one big step equals two smaller consecutive steps

Definition: Average Velocity

For any pair $r < t$ and point $\mathbf{z}_t$ along the flow path:

$$\bar{\mathbf{u}}(\mathbf{z}_t, r, t) := \frac{1}{t-r} \int_r^t \mathbf{v}(\mathbf{z}_\tau, \tau) \, d\tau. \quad (22)$$

- $\bar{\mathbf{u}}$ is an **underlying field** induced by $\mathbf{v}$ (no networks yet).
- As $r \rightarrow t$, $\bar{\mathbf{u}}(\mathbf{z}_t, r, t) \rightarrow \mathbf{v}(\mathbf{z}_t, t)$.
- Composition over intervals holds: one big step equals two smaller consecutive steps (additivity of integrals).

Preliminaries: Matrix function derivations and Jacobian-vector product

- Matrix function derivations
- Einsten notation and Einstein summation convention
- Jacobian-vector product

See wikipedia matrix calculus for more details.

MeanFlow Identity (core relation)

Starting from Eq. (21):

$$(t - r) \bar{\mathbf{u}}(\mathbf{z}_t, r, t) = \int_r^t \mathbf{v}(\mathbf{z}_\tau, \tau) d\tau. \quad (23)$$

Differentiate w.r.t. t (treating r as constant) and use the fundamental theorem of calculus:

$$\boxed{\bar{\mathbf{u}}(\mathbf{z}_t, r, t) = \mathbf{v}(\mathbf{z}_t, t) - (t - r) \frac{d}{dt} \bar{\mathbf{u}}(\mathbf{z}_t, r, t)} \quad (24)$$

where the total derivative expands to

$$\frac{d}{dt} \bar{\mathbf{u}}(\mathbf{z}_t, r, t) = \underbrace{\mathbf{v}(\mathbf{z}_t, t)}_{\frac{d\mathbf{z}_t}{dt}} \cdot \partial_{\mathbf{z}} \bar{\mathbf{u}}(\mathbf{z}_t, r, t) + \partial_t \bar{\mathbf{u}}(\mathbf{z}_t, r, t). \quad (25)$$

Interpretation: Eq. (24) links instantaneous and average velocities *without* introducing networks.

Mathematical significance: This identity enables learning average velocity fields that can be used for single-step generation.

Trainable objective

Parameterize a network $\bar{\mathbf{u}}_\theta(\mathbf{z}, r, t)$. Use the MeanFlow identity to form a target (replace $\mathbf{v}$ by sample-conditional $\mathbf{v}_t$):

$$\text{Target: } \bar{\mathbf{u}}_{\text{tgt}} = \mathbf{v}_t - (t - r) \underbrace{(\mathbf{v}_t \cdot \partial_{\mathbf{z}} \bar{\mathbf{u}}_\theta)}_{\text{JVP}} + \partial_t \bar{\mathbf{u}}_\theta. \quad (26)$$

$$\mathcal{L}(\theta) = \mathbb{E} \left\| \bar{\mathbf{u}}_\theta(\mathbf{z}_t, r, t) - \text{sg}(\bar{\mathbf{u}}_{\text{tgt}}) \right\|_2^2. \quad (27)$$

Key components:

- $\text{sg}(\cdot)$: stop-gradient; avoids higher-order differentiation
- JVP (Jacobian-vector product) computed efficiently via autodiff
- If $r = t$, the correction term vanishes $\Rightarrow$ reduces to standard Flow Matching

Mathematical insight: This objective learns to predict average velocities over time intervals, enabling single-step generation.

Pseudocode (training)

MeanFlow training algorithm:

- 1 Sample $t, r \sim \text{LogitNormal}$; set $t \leftarrow \max(r, t)$, $r \leftarrow \min(r, t)$
- 2 Sample $\mathbf{x} \sim p_{\text{data}}$, $\epsilon \sim \mathcal{N}(0, I)$; set $\mathbf{z}_t = (1 - t)\mathbf{x} + t\epsilon$, $\mathbf{v}_t = \epsilon - \mathbf{x}$
- 3 Compute $(\bar{\mathbf{u}}_\theta, \frac{d}{dt}\bar{\mathbf{u}}_\theta) = \text{JVP}(\bar{\mathbf{u}}_\theta(\cdot, r, t), (\mathbf{v}_t, 0, 1))$
- 4 Set $\bar{\mathbf{u}}_{\text{tgt}} \leftarrow \mathbf{v}_t - (t - r) \frac{d}{dt}\bar{\mathbf{u}}_\theta$
- 5 Minimize $\|\bar{\mathbf{u}}_\theta(\mathbf{z}_t, r, t) - \text{sg}(\bar{\mathbf{u}}_{\text{tgt}})\|_2^2$ (optionally with adaptive weights)

Key innovations:

- **LogitNormal sampling**: Ensures proper time interval distribution
- **JVP computation**: Efficient higher-order derivatives
- **Stop-gradient**: Prevents training instability

Sampling (1 step or few steps)

Replace the time integral by the learned average velocity:

$$\mathbf{z}_r = \mathbf{z}_t - (t - r) \bar{\mathbf{u}}_\theta(\mathbf{z}_t, r, t). \quad (28)$$

1-NFE sampling: take $(r, t) = (0, 1)$, $\mathbf{z}_1 = \epsilon$,

$$\boxed{\mathbf{x} \equiv \mathbf{z}_0 = \epsilon - \bar{\mathbf{u}}_\theta(\epsilon, 0, 1)} . \quad (29)$$

Mathematical significance: This single-step generation maintains the same theoretical guarantees as multi-step Flow Matching.

Practical advantages:

- **Speed:** Single function evaluation
- **Stability:** No accumulation of integration errors
- **Flexibility:** Can still use few-step variants by chaining intervals

CFG inside the field (still 1-NFE)

Define a CFG velocity field

$$\mathbf{v}_{\text{cfg}}(\mathbf{z}_t, t \mid c) = \omega \mathbf{v}(\mathbf{z}_t, t \mid c) + (1 - \omega) \mathbf{v}(\mathbf{z}_t, t). \quad (30)$$

Key insight: CFG can be applied directly to the average velocity field:

$$\bar{\mathbf{u}}_{\text{cfg}}(\mathbf{z}_t, r, t \mid c) = \omega \bar{\mathbf{u}}(\mathbf{z}_t, r, t \mid c) + (1 - \omega) \bar{\mathbf{u}}(\mathbf{z}_t, r, t). \quad (31)$$

Mathematical property: This maintains the 1-NFE property while enabling conditional generation.

Practical significance: MeanFlow can achieve both single-step generation and strong conditioning, combining the best of both worlds.

Outline

- 1 Mathematical motivation for Flow Matching
- 2 Connection with Diffusion Models
- 3 Conditional Paths and Velocity Targets
- 4 Flow Matching Training and Inference
- 5 Advanced Topics and Extensions
- 6 MeanFlow: Single-Step Generation
- 7 Summary and Future Directions**
- 8 References

Key Takeaways: Flow Matching

Mathematical foundations:

- Flow Matching learns velocity fields that transport probability distributions via the continuity equation
- Conditional paths provide analytic supervision without score estimation
- The method connects to optimal transport theory and diffusion models

Computational advantages:

- No score estimation during training
- Deterministic sampling with flexible step sizes
- Stable gradients and efficient training

Practical benefits:

- Fewer function evaluations than diffusion models
- Better sample quality in many domains
- Natural extension to conditional generation

MeanFlow extension: Enables single-step generation while maintaining theoretical guarantees.

Future Directions and Open Problems

Theoretical developments:

- **Path optimization:** Finding optimal conditional paths for specific data distributions
- **Convergence analysis:** Understanding when Flow Matching achieves optimal transport
- **Error bounds:** Quantifying approximation errors in finite-sample settings

Practical improvements:

- **Architecture design:** Specialized networks for different data modalities
- **Training strategies:** Adaptive loss weighting and curriculum learning
- **Sampling efficiency:** Better ODE solvers and step size selection

Applications:

- **Scientific computing:** Molecular dynamics, fluid simulation
- **Creative AI:** Music, video, and 3D content generation
- **Data augmentation:** Synthetic data for training other models

Outline

- 1 Mathematical motivation for Flow Matching
- 2 Connection with Diffusion Models
- 3 Conditional Paths and Velocity Targets
- 4 Flow Matching Training and Inference
- 5 Advanced Topics and Extensions
- 6 MeanFlow: Single-Step Generation
- 7 Summary and Future Directions
- 8 References**

Key References

Core Flow Matching papers:

- Lipman et al., *Flow Matching for Generative Modeling* (2022)
- Geng et al., *MeanFlow: A Flow Matching Method for One-Step Generation* (2025)
- Liu et al., *Learning to Generate and Transfer Data with Rectified Flow* (2022)
- Song et al., *Score-based generative modeling through stochastic differential equations* (2020)
- Albergo et al., *stochastic interpolants: a unifying framework for flows and diffusions* (2023)

Implementation resources:

- Flow Matching Guide and Code (GitHub)
- Open source libraries: diffusers
- Tutorial notebooks and examples

Welcome inputs

Any comments?

Welcome your inputs!